An-Najah National University

Faculty of Engineering and
Information Technology Database
Course Project

Virtual Hospital Management System

Prepared By:
Rahaf Azzam
Yaqeen
Sofian Hasan

Supervised
By :
Dr. Firas
Shakaa

Table of Contents

1. Introduction and Problem Description

2. Objectives and Use Cases

3. Database Design, ER Diagram, and Normalization

4. SQL Implementation

5. Java Application Implementation

6. User Interfaces and Reports

7. Business Logic, Deliverables, Challenges, and Conclusion

1. Introduction and Problem Description

Hospitals process large amounts of patient and medical information every day, including patient records, doctor schedules, appointments, visits, diagnoses, prescriptions, laboratory tests, admissions, bills, and payments. Manual management can cause duplicated records, missing data, scheduling difficulties, and delayed billing. The Virtual Hospital Management System solves these issues by organizing hospital operations in a relational MySQL database connected to a Java desktop application.

The project demonstrates a full database lifecycle: idea, requirements, ER design, normalization, SQL schema, sample data, Java/JDBC integration, CRUD operations, reports, final demo, and presentation assets.

2. Objectives and Use Cases

- Design a normalized relational database for hospital workflows.
- Implement DDL and DML scripts for schema creation and system operations.
- Build a Java Swing GUI connected to MySQL using JDBC.
- Support CRUD for patients, doctors, appointments, bills, and payments.
- Generate reports using JOIN queries for administrative use.
- Prepare the final report, ERD, normalization document, SQL scripts, database dump, code archive, GitHub URL, slides, and demo video.

3. System Features

Module	Implemented Features
Dashboard	Live counters for patients, doctors, appointments, and bills using COUNT queries.
Patients	Add, update, delete, clear, load, and select patient records in a table.
Doctors	Manage doctor information including specialty, contact details, and department ID.
Appointments	Manage appointment date, time, status, patient ID, and doctor ID.
Billing and Payments	Create bills, update bills, delete bills, add payments, and refresh bill status.
Reports	Appointments, unpaid bills, visit history, lab results, admissions, and prescription medications.

4. Database Design

The database contains medical, administrative, and financial entities. Primary keys uniquely identify records, while foreign keys connect related tables and enforce referential integrity. The design supports realistic hospital operations from appointment booking to visit records, lab tests, billing, payment, and reporting.

Main entities: Patient, Doctor, Department, Appointment, Visit, Diagnosis, Treatment, Prescription, Medication, Prescription_Medication, Laboratory_Test, Lab_Result, Bill, Payment, Room, and Admission.

Relationship	Type	Meaning
Department - Doctor	1:N	One department has many doctors.
Patient - Appointment	1:N	One patient can book many appointments.
Doctor - Appointment	1:N	One doctor can handle many appointments.
Appointment - Visit	1:1	One appointment may produce one visit.
Prescription - Medication	M:N	Handled through Prescription_Medication.
Bill - Payment	1:N	One bill can have one or more payments.

5. ER Diagram

The ER diagram shows the project entities, attributes, and relationships. It covers the complete workflow: departments and doctors, patients and appointments, visits, diagnoses, treatments, prescriptions, lab tests, admissions, rooms, bills, and payments.

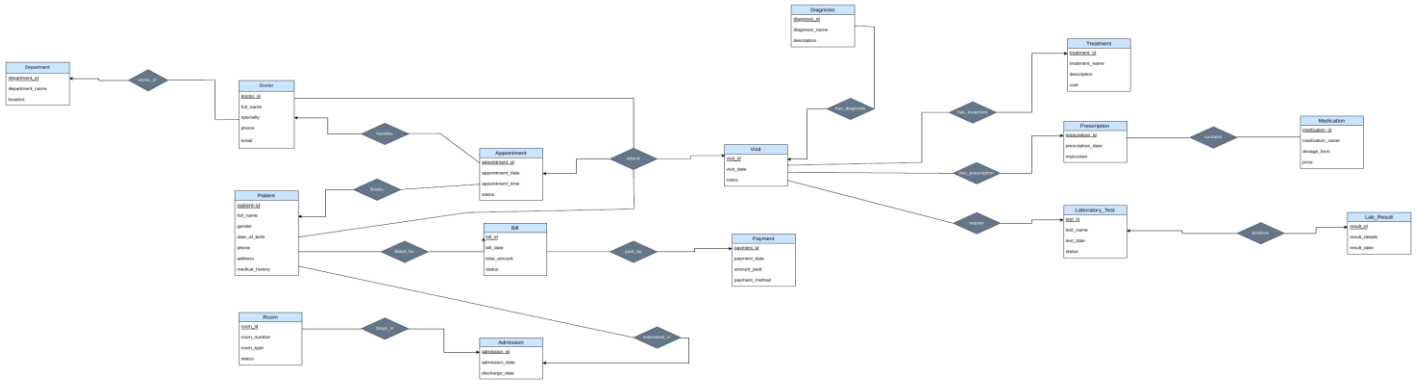


Figure 1. ER diagram of the Virtual Hospital Management System.

6. Normalization

The schema follows normalization rules to reduce redundancy and improve consistency. Patient data is separated from appointment, bill, and payment data. Doctor data is separated from department data. The many-to-many prescription-medication relationship is represented by a junction table.

Normal Form	Application in the Project
1NF	Attributes contain atomic values and no repeating groups.
2NF	Non-key attributes depend on the whole key, especially in junction tables.
3NF	department data is separated into the Department table.

7. SQL Implementation

The project uses MySQL. DDL statements create the database schema, define primary keys, and define foreign key constraints. DML statements insert sample data and support application operations such as SELECT, INSERT, UPDATE, and DELETE.

Example DDL:

```
CREATE TABLE Patient (
patient_id INT PRIMARY KEY,
full_name VARCHAR(100) NOT NULL,
gender VARCHAR(10), date_of_birth DATE,
phone VARCHAR(20), address VARCHAR(200), medical_history TEXT );
```

Example CRUD DML:

```
INSERT INTO Patient VALUES (?, ?, ?, ?, ?, ?, ?);
UPDATE Patient SET full_name=?, gender=?, date_of_birth=?, phone=?, address=?, medical_history=? WHERE patient_id=?; DELETE FROM Patient WHERE patient_id=?;
SELECT patient_id, full_name, gender, date_of_birth, phone, address, medical_history FROM Patient ORDER BY patient_id;
```

8. Java Application Implementation

The application is a Java Swing desktop system named HospitalApp. It extends JFrame and uses JTabbedPane for navigation, JTable and DefaultTableModel for displaying records, JTextField for data entry, JButton for actions, and JDBC objects such as Connection, Statement, PreparedStatement, ResultSet, and ResultSetMetaData for database interaction.

Java Component	Role
HospitalApp extends JFrame	Main application window and controller.
JTabbedPane	Separates Dashboard, Patients, Doctors, Appointments, Billing & Payments
PreparedStatement	Executes parameterized INSERT, UPDATE, and DELETE operations.
ResultSetMetaData	Creates dynamic report columns depending on the selected SQL report.
JOptionPane	Displays success, error, and confirmation messages.

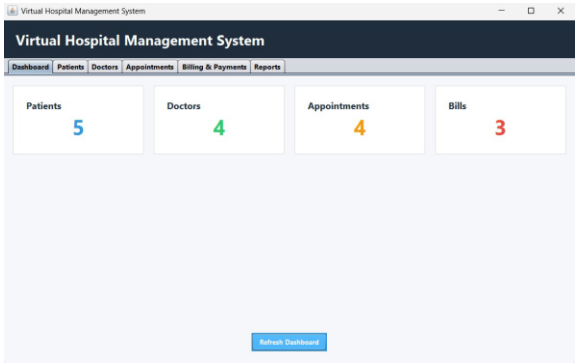
Example Java logic:

```
private void refreshDashboard() {
patientCountLabel.setText(String.valueOf(countRows("Patient")));
doctorCountLabel.setText(String.valueOf(countRows("Doctor")));
appointmentCountLabel.setText(String.valueOf(countRows("Appointment")));
billCountLabel.setText(String.valueOf(countRows("Bill")));
}
```

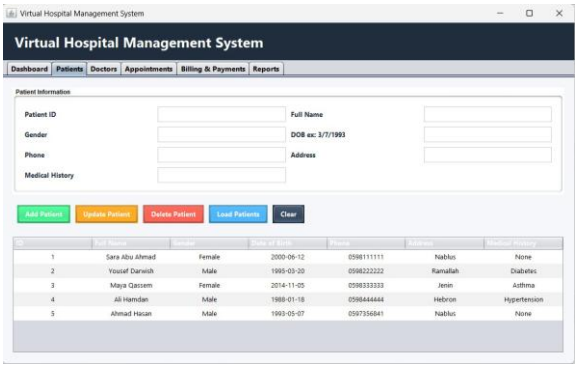
The payment method calls updateBillStatus(conn, billId), which changes a bill to Paid when total payments cover the bill amount, otherwise it remains Unpaid.

9. User Interface Screenshots

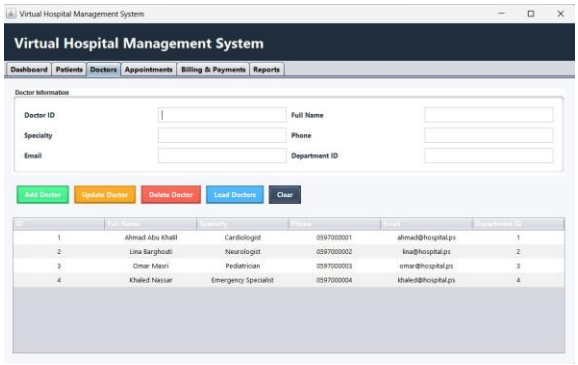
The graphical user interface allows the user to interact with the database without writing SQL manually. Each tab
provides forms, action buttons, and tables that reload data after operations.



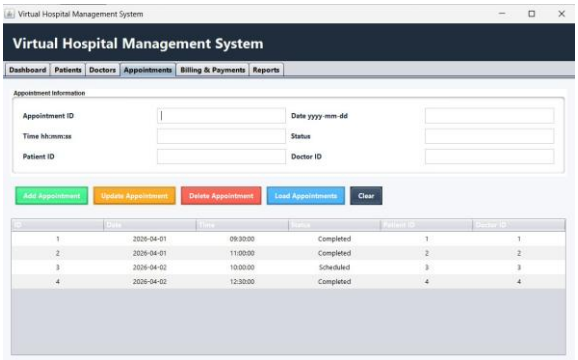
Dashboard



Patients

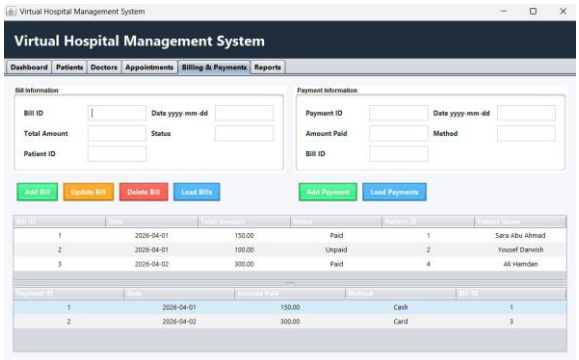


Doctors

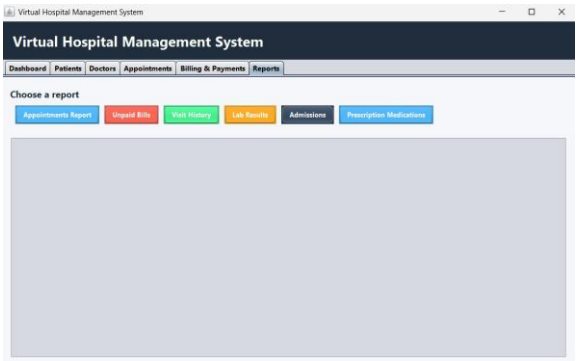


Appointments

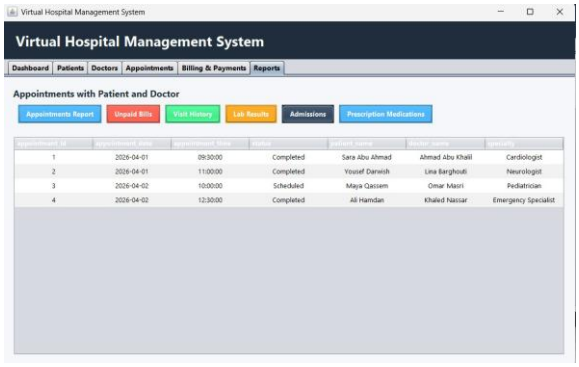
User Interface Screenshots – Continued



Billing and Payments



Reports Menu



Appointments Report

Interface Summary

The screenshots demonstrate the final Java application in action: a dashboard for totals, management tabs for CRUD operations, billing and payment handling, and reports that display joined data from multiple tables.

10. Reports, Business Logic, and Queries

The Reports tab proves that the application retrieves meaningful information, not only individual table records. It
uses SQL JOIN statements and dynamic table columns to display administrative reports.

Report	Purpose
Appointments Report	Displays appointment details with patient names, doctor names, and specialties.
Unpaid Bills	Filters bills where status is Unpaid.
Visit History	Shows patient visits with doctor information and notes.
Lab Results	Displays lab tests and their results using LEFT JOIN.
Admissions	Shows admitted patients and assigned rooms.
Prescription Medications	Displays medications connected to each prescription through the junction table

Example report query:

```
SELECT a.appointment_id, a.appointment_date, a.appointment_time, a.status, p.full_name AS
patient_name, d.full_name AS doctor_name, d.specialty FROM Appointment a
JOIN Patient p ON a.patient_id = p.patient_id
JOIN Doctor d ON a.doctor_id = d.doctor_id
ORDER BY a.appointment_date, a.appointment_time;
```

Implemented business rules:

- Dashboard counters are calculated from the database using COUNT queries.
- Tables reload after successful insert, update, and delete operations.
- Delete buttons ask for confirmation before removing records.
- Payment insertion triggers bill status recalculation.
- Date input is parsed and converted to SQL DATE values before database insertion.

11. Final Submission Assets

The final submission package includes the project report, ER Diagram PDF, normalization document, SQL scripts, database dump, GitHub repository URL, archived source code, presentation slides, and demo video.

12. Challenges

- Designing a complete hospital workflow with enough connected entities.
- Maintaining referential integrity with foreign keys.
- Implementing the many-to-many prescription-medication relationship.
- Connecting Java Swing controls to MySQL using JDBC.
- Creating flexible report tables using ResultSetMetaData.
- Handling input errors such as invalid dates or foreign key violations.

13. Future Improvements

- Add login and role-based access for admin, doctor, lab technician, and billing staff.
- Add automatic appointment conflict checking.
- Add search and filter options inside tables.
- Export reports to PDF or Excel.
- Develop a web or mobile version of the system.

14. Conclusion

The Virtual Hospital Management System demonstrates a complete database project from initial idea to final delivery. It includes ER design, normalization, SQL implementation, database sample data, Java/JDBC integration, GUI screens, CRUD operations, reports, and billing logic. The project shows how a relational database can improve hospital management and how Java can provide a practical interface for real database operations.

15. References

- Oracle Java Documentation
- MySQL Documentation
- JDBC Documentation
- Database Systems Concepts